

Assignment 1: Multi-Stage AI Path Planning and Control for Autonomous Cars and Drones.

GROUP20

Egill Pétur Ómarsson Marceau Messaussier

2001-10-30

2003-08-05

epom@kth.se

marceaum@kth.se



Abstract

This project presents a multi-stage navigation pipeline for autonomous cars and drones in complex environments. Modern vehicles are increasingly incorporating AI-powered driving assistance to improve passenger safety and comfort. This trajectory points toward fully autonomous vehicles, which introduce a new set of complex challenges, some of which this project addresses in a simulated environment. Among these challenges, reliably navigating through obstacle-dense environments remains particularly important, as even small planning or smoothing errors can lead to collisions or dynamic infeasibility. To ensure safety and efficiency, we implemented an augmented A* (A-star) algorithm incorporating Voronoi field costs and physical clearance validation. To transform discrete search results into flyable or drivable trajectories, we developed a gradient-descent smoother constrained by obstacle repulsion, path tension, and anti-curvature forces—specifically for the car—and high-order derivative forces—specifically tuned for drone dynamics. Finally, we implemented a predictive kinematic velocity profiler that pre-calculates optimal speeds based on local curvature and acceleration limits. This approach bridges the gap between high-level path planning and low-level vehicle control, resulting in robust, high-speed traversal across challenging competition maps.

1 Introduction

Path planning and motion control are vital components of autonomous driving. As vehicles become increasingly capable of operating without human input, the ability to navigate complex environments safely, reliably, and efficiently becomes a critical engineering challenge. Indeed, effective path planning algorithms must account for obstacles, physical vehicle constraints, and real-time decision-making in order to avoid deadlock situations. This project aims to create a path planning algorithm for a car and a drone and control those vehicles to follow the path through various maps. The main objective of the project was to get the vehicles to traverse the map in the fastest time, as the total team time was used to determine the winners of the final competition.

1.1 Contribution

The main contribution of this project is the design and implementation of a complete planning-and-control method that enables autonomous vehicles to efficiently navigate various maps. Specifically, the contributions of this project are as follows:

- The implementation of a path planning algorithm to compute feasible and reliable paths for both a car and a drone on different map layouts.
- The use of vehicle control strategies to ensure that the path is followed accurately.
- The integration of planning and control optimized for minimizing the traversal time.

These contributions highlight the practical challenges of autonomous navigation and demonstrate how algorithmic planning and control choices may be used in real-world scenarios.

1.2 Outline

The report first motivates the need for a unified navigation pipeline that can handle both non-holonomic (car) and holonomic (drone) motion in cluttered Unity maps, where execution time is the primary performance measure. It then summarizes the key ideas drawn from prior work—grid-based search, Voronoi-based clearance costs, trajectory smoothing, and time-aware velocity

selection—and explains how these components are combined into our three-stage pipeline. After presenting the planning stack (Voronoi A*, gradient-based smoothing, and curvature-limited velocity profiling), the report details the vehicle-specific controllers used to track the resulting trajectories: pure pursuit steering for the car and a vector-field-inspired tracker for the drone. Finally, we evaluate the system in the competition setting and discuss why the resulting trade-off between clearance, smoothness, planning cost, and execution speed led to competitive performance.

2 Related Work

Path planning and control for autonomous cars and drones requires jointly addressing collision avoidance, kinematic feasibility, and motion smoothness. Classical planners provide efficient solutions in discrete spaces, but practical deployment on real vehicles demands additional refinement stages to account for non-holonomic and dynamic constraints. This section reviews prior work that motivates the pipeline adopted in this project: discrete planning, geometric smoothing, velocity optimization and control of vehicles.

The cornerstone of grid-based planning is the A* algorithm, introduced by Hart et al. [3]. A* efficiently computes shortest paths in discretized environments and is resolution-complete. However, it assumes holonomic motion and does not incorporate vehicle-specific constraints. As a result, the generated paths typically exhibit sharp turns and discontinuities that are infeasible for steering-constrained cars and dynamically limited drones.

To overcome these limitations, Dolgov et al. [2] proposed a specific path post-processing. Their work introduced a gradient-descent-based path smoothing stage, where a multi-objective cost function reduces curvature while increasing obstacle clearance using Voronoi-based gradients. This approach demonstrates that post-processing via gradient descent is an effective way to transform a discrete A* path into a geometrically feasible trajectory. We have to note that they used a Hybrid A* instead of our A*.

For aerial robots, geometric smoothness must be complemented by higher-order dynamic consistency. Mellinger and Kumar [4] showed that quadrotor trajectories should minimize higher-order derivatives such as jerk or snap to ensure trackability and aggressive maneuvering. Their results justify the use of gradient-descent smoothing formulations that explicitly penalize jerk when generating drone trajectories, even if the initial path is obtained from a purely geometric planner.

Finally, several works on adaptive motion planning [6] emphasize that path geometry and velocity profiles cannot be treated independently. By

computing curvature-dependent velocity limits, planners can generate time-parameterized trajectories that respect either acceleration constraints or curvature constraints and improve overall performance. This perspective motivates the final stage of our approach, where ideal velocities are computed from the smoothed path.

In addition to planning, prior work on vehicle guidance motivates using different tracking controllers for cars and drones due to their distinct motion constraints. For car-like vehicles, pure pursuit [1] is a widely used geometric tracking method that computes a curvature (steering command) to reach a look-ahead point on the path, typically producing smooth arcs when the reference path is sufficiently well-formed. For aerial vehicles, guidance methods based on vector fields [5] define a desired direction of motion that combines forward progress along the path with a correction term that reduces cross-track error, enabling convergence without requiring the vehicle to follow the path using steering geometry. Together, these approaches motivate our choice of pure pursuit for the car and a vector-field-inspired tracker for the drone, with velocity regulation used to ensure smooth execution.

3 Proposed Method

Our navigation framework follows a hierarchical pipeline that transforms raw environmental data into kinematically feasible trajectories through a three-stage process: global search, gradient-based smoothing, and predictive velocity profiling followed by a pure pursuit controller for the car and a vector field inspired controller for the drone. This structure ensures that the vehicle does not merely find the shortest path but rather a trajectory that respects the differential constraints of the platform while maintaining a safety buffer.

3.1 Path Planning Stack

The path planning process is divided into a discrete search phase and a continuous optimization phase to reconcile computational efficiency with physical executability.

3.1.1 Safety-Aware Global Search (Voronoi A*)

To prioritize safety in narrow and cluttered environments, we augment the standard A* algorithm with a Voronoi-weighted cost function and adaptive clearance-based pruning. This section details how obstacle-aware costs and vehicle footprint constraints are integrated into the global search stage to produce safer and more robust paths.

The first modification concerns the A* cost computation, which now explicitly accounts for obstacles through a Voronoi field. Unlike traditional A* implementations that minimize only Euclidean distance, our cost function $G(n)$ incorporates a penalty derived from a pre-computed Voronoi field. This creates a repulsive effect from obstacles, encouraging the planner to favor the center of free space, which is particularly important for stability at higher speeds. The Voronoi field formulation follows the definition introduced in [2] and is given by:

$$\rho_V(x, y) = \frac{\alpha}{\alpha + d_O(x, y)} \cdot \frac{d_O(x, y)}{d_O(x, y) + d_V(x, y)} \cdot \frac{(d_O - d_O^{max})^2}{(d_O^{max})^2}$$

where d_O and d_V denote the distances to the nearest obstacle and to the edge of the Generalized Voronoi Diagram (GVD), respectively. The parameters $\alpha > 0$ and $d_O^{max} > 0$ control the falloff rate and the maximum effective range of the field. For $d_O > d_O^{max}$, the Voronoi cost is set to zero.

Despite the modified cost function, A* retains its optimality guarantees. We continue to use a Euclidean heuristic, which remains admissible under the augmented cost formulation [3]. As a result, the algorithm still returns an optimal path with respect to the new cost, effectively trading off path length against obstacle proximity. This explains why the resulting trajectories remain collision-free while maintaining a high safety margin.

A second key modification is the introduction of footprint-aware path pruning and a 'safe strategy' that consists of finding the widest possible path. This is particularly useful for maps with the most obstacles. Instead of simple point-collision checks, we use **SphereCast** validations to account for the physical footprint of the vehicle. If a candidate node corresponds to a configuration where the vehicle cannot fit, the path is immediately pruned. To handle challenging environments, we employ an iterative search strategy: planning starts with a large safety radius (e.g., 2.0 m) and gradually decreases it in 0.1 m steps until a feasible path is found. This ensures that the safest possible solution is always prioritized before relaxing constraints.

3.1.2 Multi-Objective Gradient Descent Smoothing

After a discrete path is obtained, we apply a basic up-sampling and then a gradient-descent-based post-processing step that transforms it into a smooth and physically feasible trajectory, suitable for either a car or a drone. This stage bridges the gap between grid-based planning and continuous execution by explicitly enforcing smoothness and vehicle-specific constraints.

The smoothing pipeline starts with a simple path up-sampling step. Once A* has been executed as described above, the resulting path is uniformly up-

sampled by inserting one additional point between each pair of consecutive nodes. As a result, a path of length n becomes a sequence of $2n - 1$ points. Through empirical evaluation, we found that a single up-sampling pass provides sufficient spatial resolution across all tested maps while keeping the computational cost low.

The core of the post-processing stage is a multi-objective gradient descent smoother. Following the formulation introduced by Dolgov et al. [2], node positions are iteratively adjusted by balancing several competing cost terms, each contributing a gradient force during optimization:

- **Voronoi and obstacle repulsion:** This term maintains a safety margin by pushing the trajectory away from obstacles. We combine a weak, long-range repulsive force derived from the Voronoi field with a strong short-range repulsion activated when the path approaches closer than a predefined threshold (typically half the vehicle radius). While the Voronoi term is usually sufficient, the additional short-range force prevents failure cases in densely cluttered environments. This formulation follows the conjugate-gradient-based optimization strategy of Dolgov [2].
- **[Car-specific] Curvature reduction:** For car-like vehicles, we minimize the local curvature of the path using the curvature cost and gradients proposed by Dolgov [2]. This explicitly tries to enforce non-holonomic feasibility and ensures compatibility with steering-based controllers.
- **[Drone-specific] High-order inertial smoothing:** For the drone, we introduce a 5-point finite-difference stencil to minimize the fourth derivative of position (snap). As shown by Mellinger and Kumar [4], penalizing high-order derivatives is crucial to avoid motor saturation and to generate differentially flat trajectories that are dynamically executable by quadrotors.

3.1.3 Kinematic Velocity Profiling

The final stage of our planning stack involves a predictive velocity profiler that pre-calculates the maximum safe speed for every segment of the trajectory based on local curvature. This stage effectively translates the geometric path into a temporal plan and makes the trajectory realizable [6].

The velocity at each node is first constrained by the local curvature κ using the relation $v(\kappa) = v_{max} \cdot (1 - \kappa/k_{max})$. To account for longitudinal dynamics, we apply a bidirectional (forward-backward) pass. The backward

pass is particularly critical: it uses the kinematic formula $v_i^2 = v_f^2 + 2ad$ to ensure the vehicle begins braking at the correct distance before a sharp turn. This predictive profiling allows the vehicle to operate at its physical limits while ensuring it never enters a turn at an unrecoverable speed.

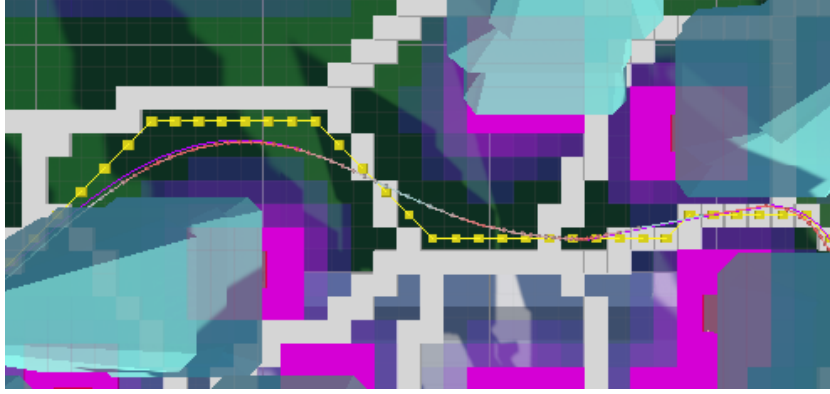


Figure 1: Overview of the path planning pipeline on Map D. The sequence begins with the global planner’s output A* (yellow), which is constrained by the grid. This is refined into a smoother trajectory, color-coded by velocity: blue indicates maximum speed, while red signifies deceleration. The underlying purple gradient represents the Voronoi field intensity, with white lines marking the Voronoi boundaries. Here Voronoi parameters are: $\alpha = 5$ and $d_O^{max} = 8$ and weights are $\omega_{voronoi} = 5$, $\omega_{snap} = 2$ and $\omega_{obstacle} = 15$.

3.2 Control Details

The control strategies used for the car and the drone differ due to their fundamentally different motion constraints. The car employs a pure pursuit controller [1] for steering, while the drone uses a path-following approach inspired by vector field guidance methods [5]. Both control strategies used PID velocity controllers to avoid jerky velocity changes and smooth out the driving of the vehicles.

3.2.1 Car controller

Due to the kinematic constraints of a car, steering is achieved by moving forward or backward while adjusting the front wheel angle. Although it is possible to control the car using a proportional or PD controller based on lateral path error, this approach often resulted in oscillations and unstable behavior when the target point lay parallel to the vehicle’s heading. Pure

pursuit resolves this issue by computing the required curvature to reach a look-ahead point on the path, producing stable steering commands even in geometrically challenging situations. This, combined with a PID velocity controller, produced a reliable path following controller that worked well for the maps made for the testing of this project.

3.2.2 Drone controller

In contrast, the drone is capable of moving freely in the horizontal plane and therefore does not require explicit heading control to follow a path. The drone’s path-following controller is divided into two components: along-path control and cross-track control. The along-path component drives the drone forward along the path by aligning motion with the local path tangent and setting a desired velocity. The cross-track component corrects lateral deviations by computing the shortest displacement from the drone to the nearest point on the piecewise linear path and applying a corrective velocity toward the path.

The combined desired velocity in the horizontal plane is then tracked using a PID controller, which generates smooth acceleration commands while respecting the drone’s physical limits. This structure reduces abrupt motion and ensures stable convergence to the path while maintaining forward progress.

3.3 Implementation Details

The proposed planning pipeline was implemented in C# within the Unity engine, using a custom grid-based spatial representation as its core abstraction. This grid component served as the central interface for both algorithmic execution and parameter tuning throughout the project.

All key planning parameters were exposed directly through the grid component inspector. This design allowed rapid iteration and systematic tuning of critical values such as the maximum velocity $V_{\max}$, the maximum admissible curvature $k_{\max}$, and the relative weights of the gradient-descent smoothing terms. Centralizing these parameters in a single component significantly simplified experimentation and ensured consistent behavior across different planning stages.

The grid resolution was fixed to $1.0f$, providing a balanced trade-off between computational efficiency and solution robustness. At this resolution, A* search remains extremely fast while still offering sufficient spatial fidelity for the subsequent smoothing and velocity computation stages, resulting in reliable and repeatable trajectories across all tested environments.

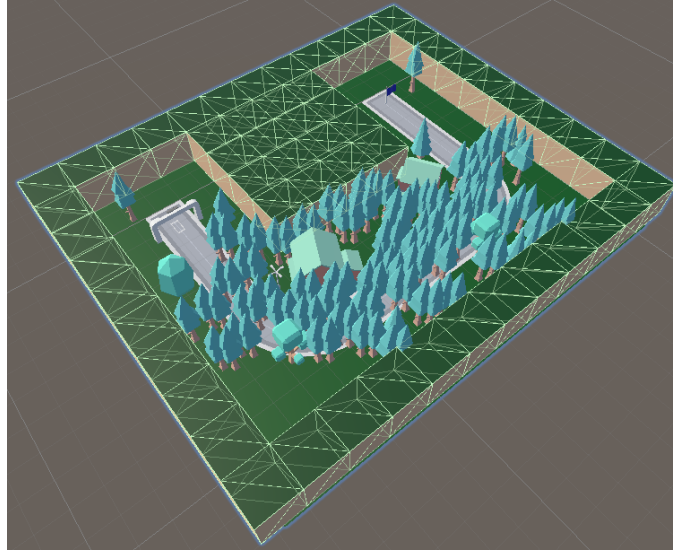


Figure 2: Map N2Hard from the challenge. This is an example of what challenges we had to face. This map is particularly dense because of the trees. The starting point was the white gate, while the goal was the blue flag.

4 Experimental results

This section first presents the Experimental Setup, which describes the competition context. It then reports the results, highlighting the efficiency and consistent performance of our Voronoi-enhanced A* strategy relative to the class average, followed by a detailed comparison with the top-ranked Group 17.

4.1 Experimental Setup

The experimental evaluation was conducted in the context of a competition, where each team developed an algorithm to autonomously drive a car from a start position to a goal. The focus of the evaluation was on execution performance rather than computational planning speed.

The objective of the competition was to safely navigate complex maps and reach a specified goal in minimal driving time. Teams were provided with a set of training maps to design, tune, and validate their planning pipelines. Once the algorithms were finalized, performance was assessed on a separate set of competition maps. See for instance Fig. 2.

4.2 Analysis of the Outcome

Our strategy represents a balanced compromise between complex kinodynamic planners and the simplicity of standard A*. In this section, we first analyze our own results before comparing our approach to the dominant strategies in the competition, with a particular focus on Group 17 (winner of the contest).

4.2.1 Our Results

Our method demonstrates strong robustness and computational efficiency. While some top-ranked groups required more than 100 seconds (see group 17, group 22) of planning on Map Z, our pipeline consistently computed a solution in under 3 seconds. Although one Hybrid A* (group 3) implementation reported sub-second planning times. Overall, our method offers a favorable trade-off between trajectory quality and computational cost.

Compared to the class average, our execution performance is consistently superior. Figure 3 shows that our driving times outperform the mean results on most maps. Similar trends are observed in the drone experiments (Fig. 4), where the combination of gradient smoothing and velocity optimization yields stable and competitive outcomes.

In summary, our pipeline provides a robust compromise between safety, smoothness, and execution speed. Although Hybrid A* achieves the best times in certain scenarios, our Voronoi-enhanced A* combined with post-smoothing delivers reliable cross-map performance with lower implementation complexity.

4.2.2 Comparative Analysis

The competition results indicate that Hybrid A* dominates the top rankings, while our Voronoi-enhanced A* remains competitive. By integrating vehicle kinematics directly into the search phase, Hybrid A* produces smoother racing lines and enables higher speeds in curves, explaining its consistent top-tier performance. See table 1 for more details concerning the comparison between our approach and the leader’s (Group 17).

Four main strategy families emerged:

- **Hybrid A* (Top-performing):** Kinematic-aware search incorporating vehicle constraints.
- **Grid-based A* (Our approach):** Deterministic search enhanced with Voronoi safety and gradient smoothing.

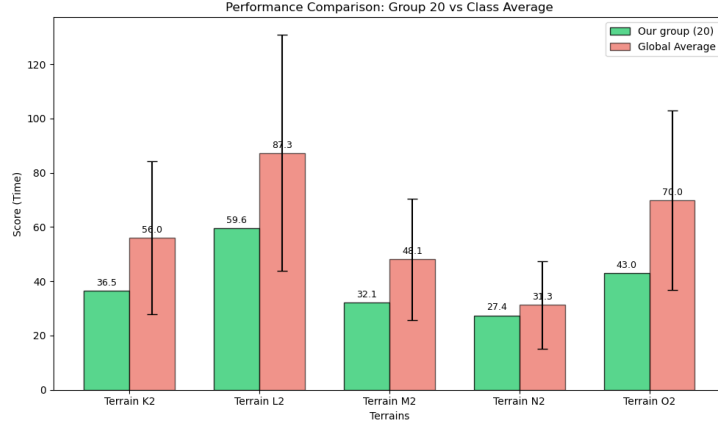


Figure 3: Comparison between our performance and the average performance of all teams. Green bars represent our performance on the map, while red bars with error bars represent the average performance taking into account the standard deviation. Note: data has been computed on the groups that completed the map.

- **RRT***: Effective in open environments but less reliable in narrow passages.
- **RL-based methods**: Promising but limited by stability and generalization issues.

Vehicle	Map	Group 20 (Ours)	Group 17 (Leader)	Class Mean
Car	K2 (Open)	36.50s (+16.5%)	31.34s	56.04s
	O2 (Hard)	43.02s (+2.8%)	41.86s	67.26s
Drone	N2 (Hard)	17.86s (+10.7%)	16.14s	30.15s

Table 1: This table shows a performance comparison on three maps and two different vehicles. Our group ends up with a reasonable loss of time compared with the top-performing Hybrid A* (Group 17).

5 Summary and Conclusions

This project presented a complete planning-and-control pipeline for autonomous navigation of a car and a drone in complex simulated environments. To address the different motion constraints of ground and aerial vehicles, the ap-

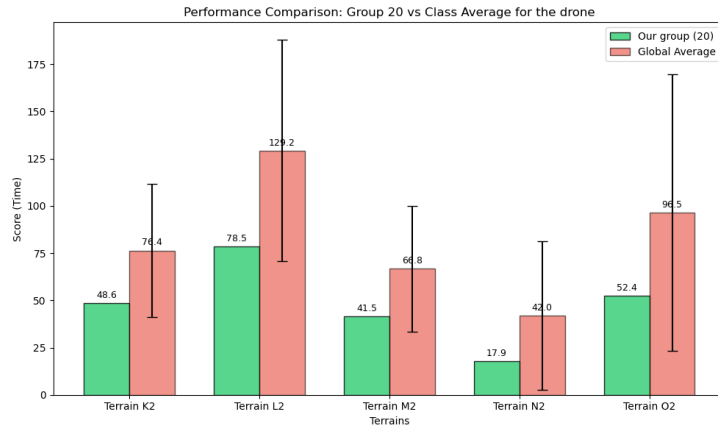


Figure 4: Comparison of our drone performance with other competing groups. The figure shows the execution time achieved on each map (green bar) by our algorithm compared with the average performance (red bars). Note: Data has been computed on the groups that completed the map.

proach combines global path planning, continuous trajectory smoothing, and vehicle-specific control strategies to achieve safe and efficient navigation.

An augmented A* planner was used to compute globally feasible paths while incorporating Voronoi-based obstacle costs and footprint-aware collision checking, which ensured adequate clearance in narrow environments. The discrete paths were then refined using a gradient-descent smoothing stage that enforced obstacle avoidance and smoothness, with curvature reduction applied for the car and high-order derivative minimization for the drone.

For control, the car employed a pure pursuit steering controller, while the drone used a vector-field-inspired path-following method with along-path and cross-track components. In both cases, PID-based velocity control ensured smooth and stable execution.

Experimental results from the competition demonstrate that the proposed pipeline achieves robust and competitive performance. While not the fastest in all scenarios, the method consistently outperformed the average competitor and required significantly less planning time than many alternative approaches. Overall, the results show that combining classical planning with continuous optimization and tailored control strategies provides an effective and practical solution for autonomous navigation of both cars and drones. One possible improvement would be using more sophisticated control strategies, as the path following component could be improved, although not necessarily in this project’s setting.

References

- [1] R. Craig Coulter. Implementation of the pure pursuit path tracking algorithm. Technical Report CMU-RI-TR-92-01, Carnegie Mellon University, The Robotics Institute, Pittsburgh, PA, 1992.
- [2] Dmitri Dolgov, Sebastian Thrun, Michael Montemerlo, and James Diebel. Practical search techniques in path planning for autonomous driving. In *Proceedings of the First International Workshop on global planning*, volume 20, pages 1–8. Stanford, CA, 2008.
- [3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [4] Daniel Mellinger and Vijay Kumar. Minimum snap trajectory generation and control for quadrotors. In *2011 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2520–2525. IEEE, 2011.
- [5] D. Nelson, B. Barber, T. McLain, and R. Beard. Vector field path following for miniature air vehicles. *IEEE Transactions on Robotics*, 23(3):519–529, 2007.
- [6] Jorge Villagra, Vicente Milanés, Joshué Pérez, and Teresa de Pedro. Smooth path and speed planning for an automated public transport vehicle. *Sensors*, 17(8), 2017.